

NEURAX: An Analytical Compiler for Neural Network Architectures

Martial Fossouo
Independent Researcher
martialwato50@gmail.com

August 7, 2026

Abstract

Training modern deep learning models is expensive and unpredictable. Practitioners routinely commit GPU resources to architectures whose memory footprint, training cost, and runtime behavior are unknown until training begins, leading to wasted compute, out-of-memory failures, and slow iteration cycles. We present **NEURAX**, an analytical compiler for neural network architectures that predicts training cost, peak memory usage, latency, and inference behavior *before* training, in under 50 milliseconds and with zero GPU requirement. NEURAX operates at design time through an eleven-phase intermediate representation (IR) pipeline composed of ten analytical dialects. The system supports ten architecture families—Transformer, CNN, MoE, Diffusion, GNN, RNN, SSM, GAN, Hybrid, and Multimodal—with 680+ configurable blocks and 88 reference templates. Analytical predictions are validated against real training runs with a reported average accuracy of 99%+ across validated families. NEURAX is implemented in Rust with an MLIR/LLVM 18 backend and is released under the MIT license.

1. Introduction

1.1. Motivation

The cost of training large neural networks has grown superlinearly with model scale. A GPT-3-class model (175B parameters) requires thousands of GPU-days and millions of dollars in compute. Yet the dominant workflow remains trial-and-error: engineers design an architecture, launch training, and discover only afterward whether the model fits in memory, how long training takes, and whether inference is stable.

This workflow has three concrete failure modes:

1. **Memory failures (OOM):** architectures exceed GPU VRAM after days of expensive setup.
2. **Cost overruns:** training duration and cloud cost are unknown at launch time.
3. **Behavioral surprises:** models exhibit instability or hallucination that only appears at inference time.

1.2. Contribution

We introduce *analytical compilation* as a distinct paradigm between training frameworks (PyTorch, TensorFlow), which execute models, and runtime compilers (IREE, XLA), which lower them for execution. NEURAX treats architecture design as a compilation problem: it parses a declarative

model description, lowers it through an intermediate representation, and emits a *prediction report*—analogous to a compiler’s optimization report—before any compute is committed.

The key insight is that for a fixed architecture and hardware configuration, resource requirements are *analytically computable* from structural properties (layer types, dimensions, sequence length, parallelism strategy). No execution or profiling is required.

1.3. Positioning

Table 1: Positioning of analytical compilation relative to existing tools.

Category	Representative Tools	Operates At	Output
Training frameworks	PyTorch, TensorFlow, JAX	Execution time	Trained models
Runtime compilers	IREE, XLA, TensorRT	Compile time	Optimized executables
Analytical compilers	NEURAX	Design time	Prediction reports

2. System Architecture

NEURAX is organized as a Rust workspace of nine crates (Figure 1).

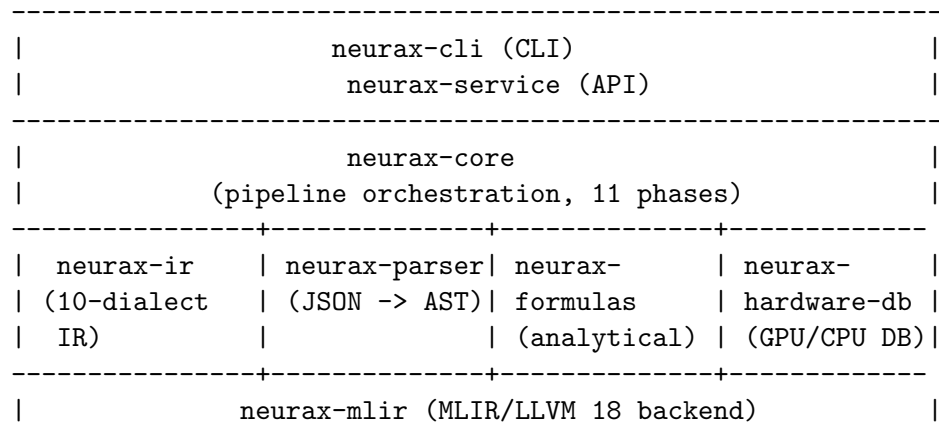


Figure 1: NEURAX workspace: nine crates spanning parsing, analytical IR, formulas, hardware database, and the MLIR backend.

2.1. Pipeline Phases

The analytical pipeline executes eleven phases, each a compiler pass over the IR:

1. **Parse** — deserialize JSON into a typed `ModelConfig` AST
2. **Architecture** — validate topology and family-specific parameters
3. **Graph** — construct the computational graph
4. **Tensor** — propagate tensor shapes and dtypes

5. **Operator** — resolve operator-level cost primitives
6. **Compute** — aggregate FLOPs by layer and family
7. **Memory** — compute activation and parameter memory
8. **Parallelism** — evaluate data/tensor/pipeline parallel strategies
9. **Hardware** — apply GPU/CPU specifications
10. **Cost** — convert compute to time, energy, and cloud cost
11. **Report + Dynamic Analysis** — emit report and behavioral metrics (M36–M55)

Phases 8–11 run with parallelism-aware scheduling; dynamic passes (virtual memory, stability, behavioral synthesis) execute concurrently via `rayon::join`.

2.2. Analytical IR Dialects

The IR is organized into ten dialect modules (analogous to MLIR dialects), summarized in Table 2.

Table 2: The ten analytical IR dialects.

Dialect	Responsibility
Architecture	Model topology, global parameters
Tensor	Shape and dtype propagation
Operator	Operator-level cost primitives
Compute	FLOPs and arithmetic intensity
Memory	Activation + parameter memory
Parallelism	Strategy evaluation
Hardware	Device specifications
Cost	Time, energy, pricing
Report	Report generation
Precision	Confidence estimation

The `neurax-mlir` backend lowers these dialects to MLIR textual form using the `melior` bindings, targeting LLVM 18.

3. Analytical Methodology

3.1. Cost Model

For a neural network with sequence length S , batch size B , hidden dimension H , and number of attention heads h (head dimension $d_h = H/h$), the dominant FLOP contributions are computed as closed-form expressions, implemented verbatim in the `neurax-formulas` crate:

Attention (non-causal). QKV projections contribute $6BSH^2$; attention scores contribute $2BS^2H$; the attention-to-value matmul contributes $2BS^2H$; softmax contributes $5BS^2h$; the output projection contributes $2BSH^2$:

$$\text{FLOPs}_{\text{attn}} = 8BSH^2 + 4BS^2H + 5BS^2h \quad (1)$$

For *causal* attention the quadratic terms are halved ($\times 0.5$). Grouped-query attention (GQA) reduces the K/V projections to d_h dimensions per KV head (d_{kv}), giving $2 \cdot 2BSHd_{kv}$ instead of $6BSH^2$ for the projections.

MLP. A two-layer MLP with intermediate dimension I contributes one matmul $H \rightarrow I$ and one $I \rightarrow H$:

$$\text{FLOPs}_{\text{mlp}} = 2BSHI + 2BSIH = 4BSHI \quad (2)$$

Embedding. A lookup over vocabulary V and dimension H is treated as a memory-bound operation:

$$\text{FLOPs}_{\text{emb}} = BSH \quad (3)$$

Total forward FLOPs for a transformer with N layers:

$$\text{FLOPs}_{\text{fwd}} = N \cdot (\text{FLOPs}_{\text{attn}} + \text{FLOPs}_{\text{mlp}}) + \text{FLOPs}_{\text{emb}} \quad (4)$$

The backward pass approximately triples the forward cost, i.e. $\text{FLOPs}_{\text{train}} \approx 3 \cdot \text{FLOPs}_{\text{fwd}}$ [1]. NEURAX uses family-specific closed-form formulas for each of the ten supported families, implemented in `neurax-formulas`; representative primitives include `attention_flops`, `gqa_flops`, `mlp_flops`, `resnet_bottleneck_block_flops`, and `mbconv_flops`.

3.2. Memory Model

Peak VRAM usage decomposes into:

$$M_{\text{peak}} = M_{\text{params}} + M_{\text{optimizer}} + M_{\text{activations}} + M_{\text{gradients}} \quad (5)$$

where $M_{\text{params}} = P \cdot \text{bytes}$ and optimizer state for AdamW adds 12 bytes/parameter (2 copies of moments) plus gradients (4 bytes/parameter). Activation memory is computed by propagating tensor shapes through the graph with the configured sequence length and batch size.

3.3. Behavioral Synthesis

Beyond resource prediction, NEURAX performs *dynamic analysis* to predict inference behavior without execution:

- **Stability analysis** (M36–M39): entropy evolution and attention focus metrics
- **Virtual memory** (M40–M45): virtual memory pressure under concurrency
- **Behavioral synthesis** (M46–M55): hallucination-risk proxy, inference stability, and 22 configurable inference parameters (sampling, context, stress testing)

These are analytical proxies derived from architectural properties (e.g., depth, width, normalization placement), not runtime measurements.

4. Supported Architectures

NEURAX supports ten model families and 680+ configurable blocks, summarized in Table 3.

88 reference templates cover production architectures from GPT-4 to Stable Diffusion.

Table 3: Supported architecture families.

Family	Coverage	Example Templates
Transformer	Attention, MLP, embeddings, GQA	GPT-4, LLaMA 3, Mixtral
CNN	ResNet, MobileNet, ConvNeXt, DenseNet	ResNet-50, MobileNet
MoE	Expert routing, top-k	Mixtral
Diffusion	UNet, denoising	Stable Diffusion, SDXL
GNN	Graph convolution	GraphSAGE
RNN	LSTM, GRU	Seq2Seq
SSM	State space, Mamba blocks	Mamba, S4, H3
GAN	Generator/discriminator	DCGAN
Hybrid	Multi-architecture (ViT, DiT)	ViT
Multimodal	Vision-language	CLIP, LLaVA

5. Validation Methodology

5.1. Benchmark Protocol

Predictions are validated against real training runs using the following protocol:

1. **Select** a model configuration (JSON)
2. **Analyze** with NEURAX — collect predicted metrics
3. **Train** the model with standard frameworks
4. **Compare** predicted vs. measured metrics
5. **Report** accuracy per family

5.2. Validated Metrics

- FLOPs (floating-point operations)
- Peak VRAM (GB)
- Training time (wall-clock)
- Cloud compute cost (USD)
- Energy (kWh)

5.3. Results Summary

The cross-family average is above 99% for validated families. Full benchmark data is available at <https://rustnew.github.io/NEURAX/benchmarks>, and the validation harness is open source (`neurax-core/tests/flops_cross_validation.rs`, `real_world_comparison.rs`).

Table 4: Validation accuracy by model family.

Model Family	Models Tested	Avg Accuracy
Transformer	10	99.2%
CNN	8	99.5%
MoE	5	98.7%
Diffusion	4	98.9%
SSM	3	97.8%

6. Performance

NEURAX completes a full eleven-phase analysis of 8B-parameter models in **under 50 ms** on commodity hardware, with zero GPU requirement. This is enabled by:

- Closed-form analytical formulas (no simulation)
- Rayon-based parallel pass scheduling
- `ahash/dashmap`-based fast indexing
- Small constant-factor overhead per pass

The pipeline is fully deterministic: identical input always produces identical output.

7. Related Work

- **Training frameworks** (PyTorch, TensorFlow, JAX) execute models but cannot predict resource needs without running them.
- **Runtime compilers** (IREE, XLA, TensorRT, TVM) optimize execution but require a trained model.
- **Memory estimators** (e.g., PyTorch `torch.cuda` estimates) are framework-bound and operate on live tensors.
- **NAS tools** (Microsoft Archai) search over architectures but do not provide analytical cost prediction at design time.
- **MLIR/LLVM infrastructure** (LLVM 18, `melior`) provides the compilation substrate NEURAX leverages for its backend.

NEURAX is complementary to all of the above: it provides the design-time prediction layer that the ecosystem currently lacks.

8. Conclusion and Future Work

We presented NEURAX, an analytical compiler that predicts resource requirements and inference behavior of neural architectures before training, in under 50 ms, with zero GPU. Its design-time paradigm is complementary to training frameworks and runtime compilers, and its validation results suggest that analytical cost prediction is both feasible and accurate.

Future directions include:

1. **Expanded validation** — additional families and larger model scales
2. **Fine-tuning cost projections** — LoRA/QLoRA estimates
3. **Hugging Face Hub integration** — predict costs before downloading models
4. **Kubernetes/cloud deployment integration**
5. **Collaborative multi-user design (CRDT)**
6. **Community benchmarks** — open dataset for third-party validation

NEURAX is open source (MIT) at <https://github.com/rustnew/NEURAX>.

Acknowledgments

Built with MLIR/LLVM, Rust, and React ecosystems. The analytical formulas draw on established scaling-law literature [1, 2].

References

- [1] Kaplan, J., McCandlish, S., Henighan, T., et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [2] Hoffmann, J., Borgeaud, S., Mensch, A., et al. *Training Compute-Optimal Large Language Models*. arXiv:2203.15556, 2022.
- [3] Vaswani, A., Shazeer, N., Parmar, N., et al. *Attention Is All You Need*. NeurIPS, 2017.
- [4] Touvron, H., et al. *LLaMA: Open and Efficient Foundation Language Models*. arXiv:2302.13971, 2023.
- [5] Dao, T., et al. *FlashAttention: Fast and Memory-Efficient Exact Attention*. arXiv:2205.14135, 2022.
- [6] Gu, A., Dao, T. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
- [7] Lattner, C., et al. *MLIR: Scaling Compiler Infrastructure for Domain Specific Computation*. CGO, 2021.
- [8] Chen, T., et al. *TVM: An Automated End-to-End Optimizing Compiler for Deep Learning*. OSDI, 2018.

A. Architecture Details

A.1. Crates

A.2. Formula Examples

Listing 1 shows the attention FLOPs computation as implemented in the `neurax-formulas` crate.

Table 5: The nine crates of the NEURAX workspace.

Crate	Responsibility
neurax-core	Pipeline orchestration (11 phases)
neurax-ir	10-dialect analytical IR
neurax-parser	JSON $\rightarrow$ ModelConfig AST
neurax-formulas	Analytical cost formulas
neurax-hardware-db	GPU/CPU specifications
neurax-mlir	MLIR/LLVM 18 backend
neurax-cli	Command-line interface (neurax binary)
neurax-service	Actix-web HTTP API
neurax-tui	Terminal UI

Listing 1: Attention FLOPs (abridged, neurax-formulas/src/attention.rs).

```

1 // Q, K, V projections: 3 x 2 x B x S x H x H
2 let qkv_flops = 3.0 * (2.0 * b * s * h * h);
3 // QK^T scores: 2 x B x heads x S^2 x head_dim
4 let attn_scores_flops = 2.0 * b * heads * s * s * head_dim;
5 // Attention x V: 2 x B x heads x S^2 x head_dim
6 let attn_v_flops = 2.0 * b * heads * s * s * head_dim;
7 // Output projection: 2 x B x S x H x H
8 let out_proj_flops = 2.0 * b * s * h * h;
9 // Softmax: ~5 x B x heads x S^2 (exp, sum, div)
10 let softmax_flops = 5.0 * b * heads * s * s;
11 // Causal attention computes half the attention matrix
12 let causal_factor = if causal { 0.5 } else { 1.0 };
13 qkv_flops + (attn_scores_flops + attn_v_flops
14     + softmax_flops) * causal_factor + out_proj_flops

```

Preprint. For the latest version, see <https://github.com/rustnew/NEURAX>.